UD4. Guións SQL e automatización de operacións

Nas sesións anteriores utilizamos sentenzas SQL para inserir, modificar e eliminar información dunha base de datos. Estas operacións executábanse normalmente de maneira individual, escribindo cada sentenza e comprobando o seu resultado.

Cando unha tarefa require executar varias operacións relacionadas, resulta interesante agrupalas nun único ficheiro ou programa SQL. Ademais, MariaDB permite incorporar variables, estruturas condicionais, bucles e procedementos almacenados para construír pequenos programas que automatizan tarefas de administración e manipulación de datos.

1. Introdución aos guións SQL

1.1. Que é un guión SQL?

Un guión SQL é un ficheiro de texto que contén unha ou varias sentenzas SQL organizadas para executarse de maneira secuencial. Habitualmente utiliza a extensión `.sql`.

Os guións permiten agrupar operacións que forman parte dunha mesma tarefa. En lugar de executar manualmente cada sentenza, podemos gardalas nun ficheiro e executar o conxunto cando sexa necesario.

Un guión SQL pode incluír:

- Creación e modificación de bases de datos e táboas.
- Inserción, actualización e eliminación de rexistros.
- Consultas para comprobar o estado dos datos.
- Creación de usuarios e asignación de permisos.
- Transaccións.
- Declaración de variables e estruturas de control.
- Definición de procedementos almacenados.

1.2. Organización dun ficheiro SQL

Un ficheiro SQL debe organizarse de maneira que as sentenzas se executen na orde correcta. Por exemplo, non podemos inserir datos nunha táboa que aínda non foi creada.

Unha estrutura habitual para un guión de inicialización sería:

```sql
-- 1. Crear a base de datos
CREATE DATABASE IF NOT EXISTS academia;

-- 2. Seleccionar a base de datos
USE academia;

-- 3. Crear as táboas
CREATE TABLE cursos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    horas INT
) ENGINE=InnoDB;

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    edad INT,
    curso_id INT,
    FOREIGN KEY (curso_id) REFERENCES cursos(id)
) ENGINE=InnoDB;

-- 4. Inserir datos
INSERT INTO cursos (nombre, horas)
VALUES
    ('DAW1', 100),
    ('DAW2', 100),
    ('DAM1', 200);

INSERT INTO alumnos (nombre, edad, curso_id)
VALUES
    ('Ana García', 20, 1),
    ('Luis López', 21, 1),
    ('Marta Fernández', 19, 2);

-- 5. Comprobar os datos
SELECT * FROM cursos;
SELECT * FROM alumnos;
```

Neste exemplo, as sentenzas execútanse seguindo unha secuencia lóxica: primeiro créase a base de datos, despois as táboas, posteriormente insírense os datos e finalmente compróbase o resultado.

1.3. Comentarios nos guións SQL

Os comentarios permiten documentar o código e explicar a finalidade das diferentes operacións. Non se executan como sentenzas SQL.

```
-- Comentario dunha soa liña

# Outra forma de comentario dunha soa liña

/*
   Comentario
   de varias liñas
*/

SELECT * FROM alumnos;
```

É recomendable utilizar comentarios para separar os bloques do guión, indicar os obxectivos de cada operación e explicar aquelas partes que poidan resultar difíciles de comprender.

1.4. Execución dun ficheiro SQL en MySQL Workbench

MySQL Workbench permite escribir, gardar e executar ficheiros SQL desde o editor de consultas.

1. Abrir MySQL Workbench e conectarse ao servidor MariaDB.
2. Crear unha nova pestana de consulta SQL.
3. Escribir ou abrir o ficheiro `.sql`.
4. Revisar a orde das sentenzas e comprobar que non existen erros.
5. Executar o guión completo ou seleccionar as sentenzas que se queren executar.
6. Revisar os resultados e as mensaxes de execución.

Tamén é posible executar un ficheiro desde a terminal de Debian mediante o cliente de MariaDB:

```
mariadb -u root -p academia < guion.sql
```

O operador `<` redirixe o contido do ficheiro á entrada do cliente MariaDB. As sentenzas execútanse no servidor utilizando a base de datos indicada no comando.

> **Importante:** un ficheiro SQL pode conter operacións destrutivas, como `DROP TABLE`, `DELETE` ou `UPDATE`. Antes de executar un guión é necesario revisar o seu contido e comprobar que se está conectado á base de datos correcta.

2. Variables e parámetros en guións SQL

2.1. Que é unha variable?

Unha variable é un espazo de memoria temporal no que se almacena un valor que pode utilizarse posteriormente durante a execución dun programa.

Nas operacións SQL habituais utilizamos valores directamente nas sentenzas. Por exemplo:

```
UPDATE alumnos
SET edad = 21
WHERE id = 1;
```

Se queremos reutilizar ese valor en varias operacións, podemos almacenalo nunha variable e referencialo posteriormente.

En MariaDB debemos distinguir entre as variables de usuario e as variables locais dos programas SQL:

Tipo	Sintaxe	Ámbito
Variable de usuario	`@nome_variable`	Na sesión actual do cliente.
Variable local	`DECLARE nome_variable`	Dentro dun bloque ou programa almacenado.

2.2. Variables de usuario

As variables de usuario permiten almacenar valores durante unha sesión de conexión ao servidor. Identifícanse mediante o símbolo `@` diante do nome.

Podemos asignarlles un valor utilizando `SET`:

```
SET @idade = 20;

SELECT @idade;
```

O resultado da consulta será o valor almacenado na variable.

Tamén podemos utilizar estas variables nas sentenzas DML:

```
SET @novo_idade = 22;
SET @alumno_id = 1;

UPDATE alumnos
SET edad = @novo_idade
WHERE id = @alumno_id;

SELECT * FROM alumnos
WHERE id = @alumno_id;
```

Neste exemplo, os valores do identificador e da nova idade almacénanse en variables e utilízanse posteriormente na actualización.

> **Características:** as variables de usuario non necesitan unha declaración previa mediante `DECLARE`. Poden utilizarse directamente e manteñen o seu valor durante a sesión de conexión.

2.3. Variables locais con DECLARE

As variables locais utilízanse dentro de programas almacenados, como procedementos e funcións, ou dentro de bloques compostos SQL. Decláranse mediante a instrución `DECLARE`.

A súa estrutura básica é:

```
DECLARE nome_variable tipo_dato [DEFAULT valor];
```

Algúns exemplos de declaracións:

```
DECLARE contador INT DEFAULT 0;
DECLARE nome_alumno VARCHAR(100);
DECLARE media_notas DECIMAL(5,2);
DECLARE activo BOOLEAN DEFAULT TRUE;
```

O tipo de dato debe ser compatible coa información que se vai almacenar. Podemos utilizar os tipos habituais de MariaDB, como:

- `INT`: números enteiros.
- `DECIMAL`: números decimais con precisión definida.
- `VARCHAR`: cadeas de caracteres.
- `DATE`: datas.

- `BOOLEAN`: valores lóxicos.

As variables locais deben declararse ao comezo do bloque `BEGIN ... END`, antes das instrucións executables.

2.4. Asignación de valores con SET

Unha vez declarada unha variable local, podemos asignarlle un valor mediante `SET`.

```
DELIMITER //

CREATE PROCEDURE exemplo_variable()
BEGIN
    DECLARE idade INT DEFAULT 0;

    SET idade = 20;

    SELECT idade;
END //

DELIMITER ;
```

O procedemento declara a variable `idade`, asígnalle o valor 20 e devolve o seu contido mediante unha consulta.

Para executar o procedemento:

```
CALL exemplo_variable();
```

2.5. Asignación mediante SELECT ... INTO

Tamén podemos almacenar nunha variable o resultado dunha consulta utilizando `SELECT ... INTO`.

```
DELIMITER //

CREATE PROCEDURE consultar_alumnos()
BEGIN
    DECLARE total INT DEFAULT 0;

    SELECT COUNT(*)
    INTO total
    FROM alumnos;

    SELECT total AS numero_alumnos;
END //

DELIMITER ;
```

A consulta calcula o número de rexistros da táboa `alumnos` e almacena o resultado na variable local `total`.

> **Atención:** unha consulta `SELECT ... INTO` destinada a unha variable escalar debe devolver unha única fila. Se non devolve ningunha, pode producirse a condición `NOT FOUND`; se devolve varias, producirase un erro.

3. Estruturas condicionais

3.1. Que son as estruturas condicionais?

As estruturas condicionais permiten executar diferentes instrucións dependendo de se se cumpre ou non unha condición.

Por exemplo, nun programa podemos comprobar se un alumno aprobou unha materia e realizar unha operación diferente en función da súa nota.

En MariaDB, dentro dos procedementos almacenados, podemos utilizar as estruturas `IF`, `ELSEIF` e `ELSE`.

3.2. Estrutura IF ... THEN

A estrutura `IF` permite executar un bloque de instrucións cando se cumpre unha condición.

```
IF condicion THEN
    instrucións;
END IF;
```

Por exemplo, podemos comprobar se un alumno aprobou:

```
DELIMITER //

CREATE PROCEDURE comprobar_nota(
    IN p_nota DECIMAL(4,2)
)
BEGIN
    IF p_nota >= 5 THEN
        SELECT 'Aprobado' AS resultado;
    END IF;
END //

DELIMITER ;
```

O procedemento recibe unha nota e devolve a mensaxe «Aprobado» cando o valor é igual ou superior a 5.

3.3. Estrutura IF ... ELSE

Se queremos definir que facer cando a condición non se cumpre, podemos engadir un bloque `ELSE`.

```
IF condicion THEN
    instrucións_se_verdadeiro;
ELSE
    instrucións_se_falso;
END IF;
```

Exemplo:

```
DELIMITER //

CREATE PROCEDURE avaliar_nota(
    IN p_nota DECIMAL(4,2)
)
BEGIN
    IF p_nota >= 5 THEN
        SELECT 'Aprobado' AS resultado;
    ELSE
        SELECT 'Suspenso' AS resultado;
    END IF;
END //

DELIMITER ;
```

Agora o procedemento contempla os dous resultados posibles: se a nota é igual ou superior a 5, devolve «Aprobado»; noutro caso, devolve «Suspenso».

3.4. Estrutura IF ... ELSEIF ... ELSE

Cando existen máis de dúas posibilidades podemos utilizar `ELSEIF` para comprobar condicións adicionais.

```
IF condicion1 THEN
    instrucións1;
ELSEIF condicion2 THEN
    instrucións2;
ELSEIF condicion3 THEN
    instrucións3;
ELSE
    instrucións_por_defecto;
END IF;
```

Exemplo de clasificación dunha nota:

```
DELIMITER //

CREATE PROCEDURE clasificar_nota(
    IN p_nota DECIMAL(4,2)
)
BEGIN
    IF p_nota >= 9 THEN
        SELECT 'Sobresaliente' AS cualificacion;
    ELSEIF p_nota >= 7 THEN
        SELECT 'Notable' AS cualificacion;
    ELSEIF p_nota >= 5 THEN
        SELECT 'Aprobado' AS cualificacion;
    ELSE
        SELECT 'Suspenso' AS cualificacion;
    END IF;
END //

DELIMITER ;
```

As condicións avalíanse na orde na que aparecen. Cando unha delas se cumpre, execútase o bloque correspondente e non se comproban as seguintes alternativas.

3.5. Chamada aos procedementos condicionais

Podemos executar os exemplos anteriores utilizando `CALL`:

```
CALL avaliar_nota(7.5);

CALL clasificar_nota(9.2);

CALL clasificar_nota(4.5);
```

Cada chamada executa o procedemento co valor indicado como parámetro de entrada.

4. Estruturas repetitivas: bucles

4.1. Concepto de bucle

Un bucle é unha estrutura que permite repetir un conxunto de instrucións varias veces. Utilízase cando unha tarefa debe executarse de maneira repetida, por exemplo, para percorrer unha serie de valores ou realizar unha operación ata que se cumpra unha condición.

MariaDB dispón de tres estruturas principais para crear bucles en programas almacenados:

- `WHILE`.
- `REPEAT`.
- `LOOP`.

4.2. Bucle WHILE

O bucle `WHILE` repite un bloque de instrucións mentres unha condición sexa verdadeira.

A súa estrutura é:

```
WHILE condicion DO
    instrucións;
END WHILE;
```

A condición compróbase antes de cada repetición. Se inicialmente é falsa, o bloque non chega a executarse.

Exemplo: mostrar os números do 1 ao 5.

```
DELIMITER //

CREATE PROCEDURE exemplo_while()
BEGIN
    DECLARE contador INT DEFAULT 1;

    WHILE contador <= 5 DO
        SELECT contador;
        SET contador = contador + 1;
    END WHILE;
END //

DELIMITER ;
```

A variable `contador` comeza co valor 1. En cada repetición móstrase o seu valor e increméntase nunha unidade. Cando chega a 6, a condición deixa de cumprirse e o bucle finaliza.

4.3. Bucle REPEAT

O bucle `REPEAT` executa as instrucións e despois comproba a condición de saída. Por este motivo, o bloque execútase polo menos unha vez.

A súa estrutura é:

```
REPEAT
    instrucións;
UNTIL condicion
END REPEAT;
```

Exemplo equivalente ao anterior:

```
DELIMITER //

CREATE PROCEDURE exemplo_repeat()
BEGIN
    DECLARE contador INT DEFAULT 1;

    REPEAT
        SELECT contador;
        SET contador = contador + 1;
    UNTIL contador > 5
    END REPEAT;
END //

DELIMITER ;
```

A diferenza do `WHILE`, a condición compróbase ao final de cada repetición. O bloque execútase antes de avaliar a condición.

4.4. Bucle LOOP

O bucle `LOOP` repite as instrucións indefinidamente ata que se executa unha instrución que interrompe o bucle.

A súa estrutura básica é:

```
nome_bucle: LOOP
    instrucións;

    IF condicion_salida THEN
        LEAVE nome_bucle;
    END IF;
END LOOP nome_bucle;
```

Exemplo:

```
DELIMITER //

CREATE PROCEDURE exemplo_loop()
BEGIN
    DECLARE contador INT DEFAULT 1;

    repetir: LOOP
        SELECT contador;

        SET contador = contador + 1;

        IF contador > 5 THEN
            LEAVE repetir;
        END IF;
    END LOOP repetir;
END //

DELIMITER ;
```

O nome `repetir` identifica o bucle. A instrución `LEAVE repetir` permite saír del cando se cumpre a condición indicada.

4.5. Comparación dos bucles

Bucle	Momento de comprobación	Execución mínima
`WHILE`	Antes de executar o bloque.	Pode non executarse ningunha vez.
`REPEAT`	Despois de executar o bloque.	Polo menos unha vez.
`LOOP`	Non incorpora unha condición de saída automática.	Polo menos unha vez, se non se abandona o bloque.

> **Atención cos bucles infinitos:** se a condición de saída nunca chega a cumprirse, o procedemento pode quedar executándose indefinidamente. É fundamental actualizar correctamente as variables que interveñen na condición.

4.6. Instrucións LEAVE e ITERATE

Dentro dos bucles podemos utilizar dúas instrucións adicionais:

- `LEAVE` : abandona inmediatamente o bucle identificado polo seu nome.

- `ITERATE`: salta ao inicio da seguinte repetición do bucle, sen executar o resto das instrucións desa iteración.

Exemplo de utilización de ITERATE:

```
DELIMITER //

CREATE PROCEDURE exemplo_iterate()
BEGIN
    DECLARE contador INT DEFAULT 0;

    repetir: LOOP
        SET contador = contador + 1;

        IF contador = 3 THEN
            ITERATE repetir;
        END IF;

        SELECT contador;

        IF contador >= 5 THEN
            LEAVE repetir;
        END IF;
    END LOOP repetir;
END //

DELIMITER ;
```

Neste exemplo, cando o contador chega a 3, `ITERATE` salta ao comezo da seguinte repetición. Por iso, o valor 3 non se mostra mediante a consulta `SELECT`.

5. Procedementos almacenados

5.1. Que é un procedemento almacenado?

Un procedemento almacenado é un programa SQL que se garda no servidor da base de datos e que pode executarse posteriormente mediante unha chamada.

A diferenza dun ficheiro SQL, que contén sentenzas que se executan cando se procesa o ficheiro, un procedemento almacenado queda definido na base de datos e pode invocarse tantas veces como sexa necesario.

Os procedementos almacenados permiten:

- Agrupar varias operacións SQL nunha única chamada.

- Reutilizar código sen ter que escribir de novo todas as sentenzas.
- Utilizar variables e estruturas de control.
- Recibir valores mediante parámetros.
- Executar operacións complexas de manipulación de datos.
- Centralizar determinadas tarefas no servidor de bases de datos.

5.2. Creación dun procedemento con CREATE PROCEDURE

Para crear un procedemento almacenado utilizamos a sentenza `CREATE PROCEDURE`.

A estrutura xeral é:

```
CREATE PROCEDURE nome_procedemento()
BEGIN
    sentenzas SQL;
END;
```

O bloque `BEGIN ... END` delimita o conxunto de instrucións que forman parte do procedemento.

Exemplo dun procedemento sinxelo:

```
DELIMITER //

CREATE PROCEDURE listar_alumnos()
BEGIN
    SELECT * FROM alumnos;
END //

DELIMITER ;
```

Este procedemento non recibe parámetros e devolve todos os rexistros da táboa `alumnos`.

5.3. O delimitador DELIMITER

Nos exemplos anteriores aparece a instrución `DELIMITER`. É importante comprender a súa finalidade para poder crear procedementos almacenados desde o cliente SQL.

Normalmente, o punto e coma (`;`) indica o final dunha sentenza SQL. Porén, dentro dun procedemento necesitamos utilizar varios puntos e coma para separar as instrucións internas.

Para evitar que o cliente interprete cada punto e coma interno como o final da definición do procedemento, cambiamos temporalmente o delimitador.

```
DELIMITER //

CREATE PROCEDURE exemplo()
BEGIN
    SELECT 'Primeira instrución';
    SELECT 'Segunda instrución';
END //

DELIMITER ;
```

Neste exemplo, o delimitador cambia de `;` a `//`. Deste xeito, os puntos e coma internos forman parte do corpo do procedemento. Unha vez finalizada a súa creación, restablecemos o delimitador habitual.

> **Importante:** `DELIMITER` é unha instrución do cliente SQL, non unha sentenza que execute o servidor MariaDB. Utilízase en clientes como a consola de MariaDB ou MySQL Workbench para delimitar correctamente os bloques de código.

5.4. Parámetros dun procedemento

Os procedementos poden recibir información mediante parámetros. Estes parámetros permiten adaptar o comportamento do procedemento aos valores proporcionados en cada chamada.

MariaDB dispón de tres tipos de parámetros:

Tipo	Significado	Utilización
`IN`	Entrada.	Recibe un valor desde a chamada.
`OUT`	Saída.	Devolve un valor ao código que chamou o procedemento.
`INOUT`	Entrada e saída.	Recibe un valor e pode devolvelo modificado.

5.5. Parámetros de entrada: IN

Os parámetros `IN` permiten enviar valores ao procedemento. Dentro del, o valor recibido pode utilizarse nas consultas e operacións.

Exemplo: buscar un alumno polo seu identificador.

```
DELIMITER //

CREATE PROCEDURE buscar_alumno(
    IN p_id INT
)
BEGIN
    SELECT *
    FROM alumnos
    WHERE id = p_id;
END //

DELIMITER ;
```

O parámetro `p_id` recibe o identificador do alumno que se quere consultar.

Para executar o procedemento:

```
CALL buscar_alumno(1);

CALL buscar_alumno(2);
```

Cada chamada executa a consulta co identificador indicado.

5.6. Parámetros de saída: OUT

Os parámetros `OUT` permiten devolver información desde o procedemento ao código que o chamou.

Por exemplo, podemos crear un procedemento que devolva o número total de alumnos:

```
DELIMITER //

CREATE PROCEDURE contar_alumnos(
    OUT p_total INT
)
BEGIN
    SELECT COUNT(*)
    INTO p_total
    FROM alumnos;
END //

DELIMITER ;
```

Para chamar o procedemento utilizamos unha variable de usuario como argumento de saída:

```
CALL contar_alumnos(@total);

SELECT @total AS numero_alumnos;
```

O procedemento calcula o número de alumnos e almacena o resultado no parámetro `p_total`. Ao finalizar a chamada, o valor queda dispoñible na variable `@total`.

5.7. Parámetros INOUT

Os parámetros `INOUT` permiten enviar un valor ao procedemento e recibir posteriormente o valor modificado.

Exemplo: incrementar un valor nunha unidade.

```
DELIMITER //

CREATE PROCEDURE incrementar(
    INOUT p_numero INT
)
BEGIN
    SET p_numero = p_numero + 1;
END //

DELIMITER ;
```

Para executar o procedemento:

```
SET @valor = 10;

CALL incrementar(@valor);

SELECT @valor;
```

A variable comeza co valor 10. O procedemento recibe ese valor, increméntao e devolve o resultado á mesma variable.

5.8. Eliminar e modificar procedementos

Para eliminar un procedemento almacenado utilizamos `DROP PROCEDURE`:

```
DROP PROCEDURE IF EXISTS buscar_alumno;
```

A cláusula `IF EXISTS` evita que se produza un erro se o procedemento non existe.

Se necesitamos cambiar a definición dun procedemento, podemos eliminalo e crealo de novo. Tamén podemos consultar os procedementos existentes:

```
SHOW PROCEDURE STATUS
WHERE Db = 'academia';
```

Para consultar a definición dun procedemento concreto:

```
SHOW CREATE PROCEDURE buscar_alumno;
```

6. Exemplo completo: procedemento para matricular un alumno

Imos combinar os conceptos anteriores para crear un procedemento que permita matricular un alumno nunha asignatura.

Supoñemos que existe unha táboa `matriculas` coa seguinte estrutura simplificada:

```
CREATE TABLE matriculas (
    alumno_id INT,
    asignatura_id INT,
    nota DECIMAL(4,2),
    PRIMARY KEY (alumno_id, asignatura_id),
    FOREIGN KEY (alumno_id) REFERENCES alumnos(id),
    FOREIGN KEY (asignatura_id) REFERENCES asignaturas(id)
) ENGINE=InnoDB;
```

O procedemento recibirá o identificador do alumno e o identificador da asignatura. Antes de inserir a matrícula, comprobará que non exista xa unha matrícula para esa combinación.

```
DELIMITER //

CREATE PROCEDURE matricular_alumno(
    IN p_alumno_id INT,
    IN p_asignatura_id INT
)
BEGIN
    DECLARE existe INT DEFAULT 0;

    SELECT COUNT(*)
    INTO existe
    FROM matriculas
    WHERE alumno_id = p_alumno_id
      AND asignatura_id = p_asignatura_id;

    IF existe = 0 THEN

        INSERT INTO matriculas (
            alumno_id,
            asignatura_id,
            nota
        )
        VALUES (
            p_alumno_id,
            p_asignatura_id,
            NULL
        );

        SELECT 'Matrícula realizada correctamente' AS resultado;

    ELSE

        SELECT 'O alumno xa está matriculado' AS resultado;

    END IF;
END //

DELIMITER ;
```

O procedemento realiza as seguintes operacións:

1. Recibe os identificadores do alumno e da asignatura.
2. Declara a variable local `existe`.
3. Consulta cantas matrículas existen para esa combinación.
4. Se o resultado é cero, insire a nova matrícula.
5. Se xa existe unha matrícula, non realiza a inserción.

6. Devolve unha mensaxe indicando o resultado da operación.

Para executalo:

```
CALL matricular_alumno(1, 2);
```

A restrición de clave primaria composta tamén impide que se repita a combinación de alumno e asignatura. A comprobación previa permite, ademais, devolver unha mensaxe máis clara antes de tentar inserir un rexistro duplicado.

> **Importante:** este exemplo comproba que a matrícula non existe, pero non verifica expresamente que o alumno e a asignatura existan. As claves foráneas son as encargadas de garantir a integridade referencial durante a inserción.

7. Boas prácticas na creación de guións e procedementos

Os guións e procedementos almacenados permiten automatizar tarefas, pero tamén poden provocar cambios importantes na base de datos. É recomendable seguir unha serie de pautas:

- **Organizar o código:** separar as operacións en bloques e utilizar comentarios para explicar a súa finalidade.
- **Utilizar nomes claros:** escoller nomes descritivos para variables, parámetros e procedementos.
- **Comprobar as condicións:** revisar especialmente as condicións que controlan actualizacións e eliminacións.
- **Evitar bucles infinitos:** asegurarse de que as variables utilizadas nas condicións se actualizan correctamente.
- **Controlar os erros:** prever que pode ocorrer se unha consulta non devolve resultados ou se unha operación viola unha restrición.
- **Protexer os datos:** utilizar transaccións cando varias operacións deban realizarse como unha unidade.
- **Probar os procedementos:** executalos con diferentes valores, incluídos os casos límite e os valores incorrectos.
- **Documentar o código:** indicar que parámetros recibe o procedemento, que operación realiza e que resultados devolve.

8. Resumo das principais instrucións

Instrución	Finalidade
`DECLARE`	Declarar unha variable local.
`SET`	Asignar un valor a unha variable.
`SELECT ... INTO`	Gardar o resultado dunha consulta nunha variable.
`IF ... THEN`	Executar instrucións se se cumpre unha condición.
`ELSEIF`	Comprobar unha condición alternativa.
`ELSE`	Executar instrucións cando non se cumpre ningunha condición anterior.
`WHILE`	Repetir instrucións mentres unha condición sexa verdadeira.
`REPEAT`	Repetir instrucións ata que se cumpra unha condición.
`LOOP`	Crear un bucle que se interrompe mediante `LEAVE`.
`LEAVE`	Saír dun bucle.
`ITERATE`	Comezar a seguinte repetición dun bucle.
`CREATE PROCEDURE`	Crear un procedemento almacenado.
`CALL`	Executar un procedemento almacenado.
`DROP PROCEDURE`	Eliminar un procedemento almacenado.

9. Guión de comprobación

Antes de dar por finalizado un guión SQL ou un procedemento almacenado, convén revisar os seguintes aspectos:

1. Comprobar que se selecciona a base de datos correcta.
2. Revisar que as sentenzas están organizadas na orde adecuada.
3. Comprobar que as variables están declaradas co tipo de dato correcto.

4. Verificar que as condicións `IF` contemplan os casos necesarios.
5. Comprobar que os bucles teñen unha condición de saída.
6. Verificar que os parámetros de entrada e saída están correctamente definidos.
7. Revisar que as operacións DML afectan aos rexistros esperados.
8. Executar o procedemento con diferentes valores de entrada.
9. Comprobar os resultados e as mensaxes devoltas.
10. Verificar que non se producen erros de integridade referencial.

> **Conclusión:** os guións SQL permiten organizar e executar varias sentenzas de maneira secuencial. As variables, as estruturas condicionais e os bucles engaden capacidade de programación, mentres que os procedementos almacenados permiten gardar e reutilizar tarefas complexas directamente no servidor MariaDB.